System Design

Everything You Need to Know with Real Examples, Interview Questions & Answers

Creator & Author

Mohammad Faiz

Year 2026

Table of Contents

Right-click the TOC and select "Update Field" to refresh page numbers

Part 1: Foundations of System Design....3

What is System Design?....3

Key Concepts and Terminology....4

Scalability Fundamentals....6

Part 2: Core System Design Components....8

Load Balancers....8

Databases and Storage....10

Caching Strategies....13

Part 3: Advanced Topics....16

Microservices Architecture....16

Message Queues....19

CDNs and Global Distribution....22

Part 4: Real-World Examples....25

Designing URL Shortener....25

Designing Twitter/X Feed....28

Designing Uber/Lyft....31

Part 5: Interview Preparation....35

Common Interview Questions....35

System Design Interview Framework....40

Practice Problems with Solutions....45

Part 1: Foundations of System Design

What is System Design?

System Design is the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements. It is a critical skill for software engineers, especially those working on large-scale applications.

Think of system design like planning a city. Before building roads, houses, and utilities, city planners need to think about how many people will live there, what services they will need, and how everything will connect together. Similarly, when we design software systems, we need to plan for users, data, and how different parts of the system will work together.

Why is System Design Important?

- Scalability: Systems need to handle growth in users and data
- Reliability: Systems must work correctly even when parts fail
- Performance: Systems need to respond quickly to user requests
- Cost Efficiency: Well-designed systems use resources wisely
- Maintainability: Systems should be easy to update and fix

The System Design Process

0. Understand Requirements: Clarify functional and non-functional requirements
1. Estimate Scale: Calculate traffic, storage, and bandwidth needs
2. Design High-Level: Create a basic architecture diagram
3. Deep Dive: Detail each component and its interactions
4. Identify Bottlenecks: Find and address potential problems
5. Discuss Trade-offs: Evaluate different design choices

Key Concepts and Terminology

Latency vs Throughput

Latency is the time it takes for a single operation to complete. Imagine ordering food at a restaurant - latency is how long you wait from ordering to receiving your food. Lower latency means faster response times.

Throughput is the number of operations a system can handle in a given time period. Using the

restaurant example, throughput is how many meals the kitchen can prepare per hour. Higher throughput means the system can handle more work.

Metric	Definition	Example
Latency	Time for one request	200ms page load time
Throughput	Requests per second	10,000 requests/second
Availability	Uptime percentage	99.99% ("four nines")
Consistency	Data uniformity across system	All users see same data

Horizontal vs Vertical Scaling

Vertical Scaling means adding more power (CPU, RAM) to a single server. It is like moving to a bigger, more powerful computer. This is simple but has limits - you can only make one machine so powerful.

Horizontal Scaling means adding more servers to distribute the load. It is like hiring more workers instead of making one worker stronger. This is more complex but allows virtually unlimited scaling.

CAP Theorem

The CAP Theorem states that a distributed system can only guarantee two out of three properties:

- Consistency: All nodes see the same data at the same time
- Availability: Every request receives a response
- Partition Tolerance: System continues working despite network failures

In practice, network partitions are inevitable, so systems must choose between CP (Consistency + Partition Tolerance) or AP (Availability + Partition Tolerance).

Scalability Fundamentals

Understanding Load

Before designing any system, you must understand the load it will handle. Consider these factors:

- Daily Active Users (DAU): How many people use your system each day
- Peak Traffic: Maximum users at any given moment
- Read/Write Ratio: How many reads vs writes your system handles
- Data Growth: How fast your data grows over time

Back-of-the-Envelope Calculations

These quick calculations help estimate system requirements:

> **Example: Estimating Storage for a Photo App**
> If your app has:
> - 1 million users
> - Each user uploads 10 photos/month
> - Each photo is 5MB
>
> Monthly storage needed:
> 1,000,000 users x 10 photos x 5MB = 50TB/month
>
> Yearly storage: 50TB x 12 = 600TB/year

Single Point of Failure (SPOF)

A Single Point of Failure is any component that, if it fails, will stop the entire system from working. Good system design eliminates SPOFs through redundancy and failover mechanisms.

Examples of SPOFs include: a single database server, one load balancer, or a critical service with no backup.

Part 2: Core System Design Components

Load Balancers

A Load Balancer distributes incoming network traffic across multiple servers. Think of it as a traffic cop directing cars to different lanes to prevent congestion.

Why Use Load Balancers?

- Distribute traffic evenly across servers
- Prevent any single server from being overwhelmed
- Provide high availability through failover
- Enable horizontal scaling by adding more servers
- Handle SSL termination to reduce server load

Types of Load Balancing Algorithms

Algorithm	How It Works	Best For
Round Robin	Cycles through servers in order	Equal-capacity servers
Least Connections	Sends to server with fewest active connections	Varying request times
IP Hash	Uses client IP to determine server	Session persistence
Weighted Round Robin	Considers server capacity	Different server sizes

Layer 4 vs Layer 7 Load Balancing

Layer 4 (Transport Layer) load balancers work with TCP/UDP traffic without looking at the content. They are faster but less intelligent.

Layer 7 (Application Layer) load balancers can inspect HTTP headers, cookies, and URLs. They are slower but can make smarter routing decisions.

> **Real-World Example: E-commerce Site**
> An online store uses Layer 7 load balancing to:
> - Route /api requests to API servers
> - Route /static requests to CDN
> - Route /checkout to dedicated payment servers

> - Use session cookies to keep users on same server

Databases and Storage

SQL vs NoSQL Databases

SQL databases (like MySQL, PostgreSQL) use structured tables with predefined schemas. They are great for complex queries and ensuring data consistency.

NoSQL databases (like MongoDB, Cassandra) are more flexible and can handle unstructured data. They scale horizontally and are good for high-write workloads.

Feature	SQL	NoSQL
Schema	Fixed, predefined	Flexible, dynamic
Scaling	Vertical (mostly)	Horizontal
Best For	Complex queries, ACID	High write throughput
Examples	MySQL, PostgreSQL	MongoDB, Cassandra, Redis

Database Sharding

Sharding splits a large database into smaller, more manageable pieces called shards. Each shard contains a subset of the data.

Common sharding strategies include:

- Hash-based: Distribute based on hash of a key
- Range-based: Split by ranges (A-M, N-Z)
- Geographic: Shard by user location

Database Replication

Replication creates copies of data across multiple servers for reliability and performance.

- Master-Slave: One master handles writes, slaves handle reads
- Master-Master: Multiple masters can handle writes (more complex)
- Synchronous: All copies updated before confirming write
- Asynchronous: Write confirmed immediately, copies updated after

Indexing for Performance

Indexes are like book indexes - they help find data quickly without scanning everything. However, they slow down writes and use extra storage.

> **Indexing Example**
> Without index: Finding a user by email requires scanning all rows. O(n) time.
>
> With B-tree index: Finding a user by email uses the index tree. O(log n) time.
>
> For 1 million users: Scanning takes ~1M operations, index takes ~20 operations.

Caching Strategies

Caching stores copies of frequently accessed data in fast memory to reduce database load and improve response times.

Cache Placement Strategies

- Client-side: Browser cache, mobile app cache
- CDN: Cache static content at edge locations
- Application: In-memory cache within app servers
- Database: Database query cache

Cache Invalidation Strategies

Strategy	How It Works	Trade-offs
TTL (Time To Live)	Data expires after set time	Simple, may serve stale data
Write-Through	Update cache when writing to DB	Always consistent, slower writes
Write-Back	Write to cache first, async to DB	Fast writes, risk of data loss
Cache-Aside	App manages cache explicitly	Flexible, more complex code

Cache Eviction Policies

When cache is full, these policies decide what to remove:

- LRU (Least Recently Used): Remove items not accessed longest

- LFU (Least Frequently Used): Remove least accessed items
- FIFO (First In First Out): Remove oldest items
- Random: Remove random items (simple but less effective)

Redis Caching Example

Popular apps use Redis to cache:
- User sessions (fast login validation)
- Product catalogs (reduce DB load)
- Leaderboards (real-time rankings)
- Rate limiting counters

Part 3: Advanced Topics

Microservices Architecture

Microservices architecture breaks a large application into small, independent services that communicate through APIs. Each service handles one specific business capability.

Microservices vs Monolith

Aspect	Monolith	Microservices
Deployment	Single unit	Independent services
Scaling	Scale entire app	Scale individual services
Technology	One stack	Different stacks per service
Complexity	Simpler initially	More complex, but manageable
Team Size	Small to medium	Large, distributed teams

Service Discovery

In microservices, services need to find each other dynamically. Service discovery tools like Consul, Eureka, or Kubernetes DNS help services locate and communicate with each other.

- Service Registry: Central database of available services
- Health Checks: Monitor which services are healthy
- Load Balancing: Distribute requests across service instances

API Gateway

An API Gateway is a single entry point for all client requests. It handles:

- Request routing to appropriate microservices
- Authentication and authorization
- Rate limiting and throttling
- Request/response transformation
- SSL termination

Message Queues

Message queues enable asynchronous communication between services. Instead of calling another service directly, you send a message to a queue and the receiver processes it when ready.

Benefits of Message Queues

- Decoupling: Services do not need to know about each other
- Reliability: Messages are not lost if a service is down
- Scalability: Process messages at your own pace
- Buffering: Handle traffic spikes gracefully

Common Message Queue Systems

System	Best For	Key Features
RabbitMQ	General purpose	Reliable, flexible routing
Kafka	High throughput	Distributed, persistent, streaming
SQS	AWS integration	Managed, scalable, simple
Redis	Simple queues	Fast, in-memory, lightweight

Message Patterns

- Point-to-Point: One producer, one consumer
- Publish-Subscribe: One producer, many consumers
- Request-Reply: Send request, wait for response

Real-World Example: Order Processing

When a customer places an order:

1. Order service saves order to database
2. Order service publishes "OrderCreated" event to queue
3. Payment service processes payment asynchronously
4. Inventory service updates stock levels
5. Email service sends confirmation email

Each service works independently, improving reliability.

CDNs and Global Distribution

A Content Delivery Network (CDN) is a geographically distributed network of servers that delivers content faster by serving it from locations closer to users.

How CDNs Work

When a user requests content, the CDN routes them to the nearest edge server. If the content is cached there, it is served immediately. If not, the edge server fetches it from the origin and caches it for future requests.

CDN Benefits

- Reduced Latency: Content served from nearby locations
- Lower Origin Load: Cache hits reduce origin server traffic
- DDoS Protection: CDNs can absorb attack traffic
- Global Reach: Serve users worldwide efficiently

CDN Caching Strategies

Strategy	Description	Use Case
Static Content	Cache images, CSS, JS	All websites
Dynamic Content	Cache API responses with short TTL	API acceleration
Video Streaming	Cache video segments	Netflix, YouTube
Whole Site	Cache entire HTML pages	News sites, blogs

Popular CDN Providers

- Cloudflare: Security-focused with free tier
- AWS CloudFront: Deep AWS integration
- Akamai: Enterprise-grade, largest network
- Fastly: Real-time purging, developer-friendly

Part 4: Real-World Examples

Designing a URL Shortener (like bit.ly)

Requirements

- Functional: Shorten URLs, redirect to original, custom aliases
- Non-functional: Low latency, high availability, scalable

Estimation

Metric	Estimate
New URLs/day	10 million
Read:Write Ratio	100:1
Total URLs (5 years)	18 billion
Storage needed	~200GB (metadata only)

High-Level Design

The system has these main components:

- Load Balancer: Distributes traffic
- Application Servers: Handle API requests
- Database: Stores URL mappings
- Cache: Stores popular URLs
- Key Generation Service: Creates unique short codes

URL Shortening Algorithm

Two main approaches:

- Hash-based: Use MD5/Base62 of original URL. Risk of collisions.
- Base62 Encoding: Convert auto-increment ID to Base62. No collisions, predictable.

> **Base62 Encoding Example**
> Short URL format: bit.ly/abc123

```
1. Get next ID from database (e.g., 1000000)
2. Convert to Base62: 1000000 = '15FTG'
3. Store mapping: '15FTG' -> 'https://example.com/very/long/url'

Base62 uses: a-z, A-Z, 0-9 (62 characters)
```

Designing Twitter/X Feed

Requirements

- Post tweets (text, images, videos)
- Follow/unfollow users
- View timeline (chronological feed)
- Search tweets

Estimation

Metric	Estimate
Daily Active Users	200 million
Tweets per day	500 million
Timeline views/day	20 billion
Average followers	500

Fan-out Problem

When a user tweets, it must appear in all followers' timelines. This is called fan-out. Two approaches exist:

Approach 1: Fan-out on Write (Push)

When user tweets, immediately push to all followers' timelines.

- Pros: Fast read (timeline is pre-computed)
- Cons: Slow write, high storage for celebrities

Approach 2: Fan-out on Read (Pull)

When user views timeline, fetch tweets from all followed users.

- Pros: Fast write, no duplication
- Cons: Slow read, complex queries

Hybrid Approach

Most real systems use a hybrid:

- Normal users: Fan-out on write (push)
- Celebrities (>1M followers): Fan-out on read (pull)
- This balances read and write performance

Designing Uber/Lyft

Requirements

- Real-time driver location updates
- Match riders with nearby drivers
- Calculate ETA and pricing
- Track trip progress

Unique Challenges

- High write throughput: Drivers update location every few seconds
- Geospatial queries: Find drivers within X miles of rider
- Real-time requirements: Updates must propagate quickly

Location Tracking Architecture

The system uses these components:

- Mobile Apps: Send location updates via WebSocket
- API Gateway: Route updates to appropriate region
- Kafka: Buffer location updates for processing
- Geospatial Database: Store and query driver locations

Geospatial Indexing

To find nearby drivers efficiently:

- Geohash: Encode latitude/longitude as string prefix
- Quadtree: Divide map into hierarchical grids

- Google S2: Spherical geometry library

> **Geohash Example**
> Location: 40.7484, -73.9857 (NYC - Empire State Building)
>
> Geohash levels:
> - dr5r: ~20km x 40km area
> - dr5ru: ~5km x 5km area
> - dr5rux: ~100m x 100m area
>
> To find nearby drivers, query same geohash prefix.

Matching Algorithm

When a rider requests a ride:

6. Get rider's geohash
7. Query geospatial index for nearby available drivers
8. Filter by criteria (car type, rating, etc.)
9. Rank by ETA and distance
10. Send request to top N drivers
11. First driver to accept gets the ride

Part 5: Interview Preparation

Common Interview Questions

System design interviews test your ability to design scalable, reliable systems. Here are the most common questions you will encounter.

Beginner Level Questions

Q1: What is the difference between horizontal and vertical scaling?
Vertical scaling means adding more power (CPU, RAM) to a single server. Horizontal scaling means adding more servers to distribute the load. Horizontal scaling is preferred for large systems because it provides better fault tolerance and unlimited growth potential.

Q2: Explain the CAP Theorem.
CAP Theorem states that a distributed system can only guarantee two out of three: Consistency (all nodes see same data), Availability (every request gets a response), and Partition Tolerance (system works despite network failures). Since network partitions are inevitable, systems choose between CP (consistent but may be unavailable) or AP (always available but may be inconsistent).

Q3: What is a load balancer and why do we need it?
A load balancer distributes incoming traffic across multiple servers. It prevents any single server from being overwhelmed, provides high availability through failover, enables horizontal scaling, and can handle SSL termination. Common algorithms include Round Robin, Least Connections, and IP Hash.

Q4: What is caching and what are its benefits?
Caching stores frequently accessed data in fast memory to reduce database load and improve response times. Benefits include faster data access, reduced database load, better user experience, and cost savings. Common strategies include browser cache, CDN cache, application cache (Redis), and database query cache.

Intermediate Level Questions

Q5: How would you design a rate limiter?
A rate limiter controls how many requests a user can make in a time period. Common algorithms:

Token Bucket: Tokens added at fixed rate, each request consumes a token. Allows bursts.

Sliding Window: Track requests in time window, reject if limit exceeded. More accurate.

Fixed Window: Simple counter per time window. Can have burst issues at window boundaries.

Implementation: Use Redis with atomic operations for distributed rate limiting.

Q6: Explain database sharding and its challenges.

Sharding splits a large database into smaller pieces (shards) distributed across servers. Challenges include:

1. Data Distribution: Choosing the right shard key to avoid hotspots
2. Joins: Cross-shard joins are expensive or impossible
3. Rebalancing: Moving data when adding/removing shards
4. Unique IDs: Generating globally unique identifiers
5. Query Routing: Finding which shard contains the data

Strategies: Hash-based, range-based, or directory-based sharding.

Q7: How do you handle session management in a distributed system?

Options for session management:

Sticky Sessions: Route user to same server. Simple but loses sessions if server fails.

Session Replication: Copy sessions to all servers. High memory overhead.

Centralized Session Store: Use Redis/Memcached. Recommended approach - scalable and reliable.

JWT Tokens: Store session in client-side token. Stateless but tokens cannot be revoked easily.

Q8: What is the difference between SQL and NoSQL databases? When would you use each?

SQL databases use structured tables with predefined schemas, support ACID transactions, and are best for complex queries and data consistency. Use for: financial systems, inventory management, relational data.

NoSQL databases are schema-flexible, scale horizontally, and handle unstructured data. Types include document (MongoDB), key-value (Redis), column-family (Cassandra), and graph (Neo4j). Use for: high write throughput, flexible schemas, big data, real-time analytics.

Advanced Level Questions

Q9: Design a distributed unique ID generator.

Requirements: Unique, sortable by time, distributed generation. Solutions:

UUID: 128-bit, guaranteed unique but not sortable, large size.

Twitter Snowflake: 64-bit ID with timestamp (41 bits), machine ID (10 bits), sequence (12 bits). Sortable, distributed, compact.

Database Sequence: Use database auto-increment. Single point of failure, not distributed.

Recommended: Snowflake-like approach for most use cases.

Q10: How would you design a search autocomplete system?

Autocomplete suggests completions as user types. Design:

1. Data Collection: Log search queries, aggregate by frequency
2. Trie Data Structure: Store prefixes for fast prefix matching
3. Top K: Store top N suggestions per prefix
4. Caching: Cache popular prefixes in Redis
5. Ranking: Consider frequency, recency, personalization

Optimization: Pre-compute top suggestions, use edge caching (CDN).

Q11: Explain eventual consistency and when to use it.

Eventual consistency means that if no new updates are made, eventually all replicas will converge to the same value. It is a relaxation of strong consistency for better availability and performance.

Use when: High availability is critical, temporary inconsistency is acceptable, system can handle conflicts.

Examples: DNS, Amazon DynamoDB, Cassandra, social media likes/shares.

Conflict resolution: Last-write-wins, vector clocks, application-level merge.

Q12: How do you design for disaster recovery?

Disaster recovery strategies:

RPO (Recovery Point Objective): Maximum acceptable data loss. Achieve through frequent backups, replication.

RTO (Recovery Time Objective): Maximum acceptable downtime. Achieve through automation, standby systems.

Multi-Region: Deploy in multiple geographic regions.

Backup Strategies: Regular snapshots, point-in-time recovery, cross-region replication.

Testing: Regularly test failover procedures.

System Design Interview Framework

Follow this 4-step framework for any system design interview:

Step 1: Understand Requirements (2-3 minutes)

- Ask clarifying questions about functional requirements
- Identify non-functional requirements (scale, performance)
- Define the scope - what is in and out of scope

> **Example Questions**
> - How many daily active users?
> - What is the read/write ratio?
> - What is the expected latency?
> - Do we need real-time updates?
> - What is the budget/complexity trade-off?

Step 2: Back-of-the-Envelope Estimation (2-3 minutes)

- Calculate traffic (requests per second)
- Estimate storage requirements
- Calculate bandwidth needs
- These numbers guide your design decisions

Step 3: High-Level Design (10-15 minutes)

- Draw the basic architecture diagram
- Identify main components: load balancer, app servers, database, cache
- Explain data flow through the system
- Discuss API design briefly

Step 4: Deep Dive (10-15 minutes)

- Focus on the most critical component
- Discuss database schema and indexing
- Address bottlenecks and scaling challenges
- Talk about trade-offs in your design
- Consider failure scenarios and recovery

Common Mistakes to Avoid

- Starting to design without understanding requirements
- Ignoring non-functional requirements
- Over-engineering the solution
- Not discussing trade-offs
- Ignoring failure scenarios
- Being too quiet - think aloud!

Practice Problems with Solutions

Problem 1: Design a Web Crawler

Design a system to crawl the entire web and build a search index.

Solution Approach:

- URL Frontier: Queue of URLs to crawl, distributed across multiple servers
- HTML Fetcher: Download pages, respect robots.txt, rate limiting
- Content Parser: Extract links and content from HTML
- Duplicate Detector: Use URL hashing to avoid re-crawling
- Content Storage: Store raw HTML in distributed file system
- Index Builder: Create inverted index for search

Problem 2: Design a Chat Application

Design WhatsApp/Slack - real-time messaging with groups and media.

Solution Approach:

- WebSocket Connections: Maintain persistent connections for real-time delivery
- Message Queue: Buffer messages for offline users
- Presence Service: Track user online/offline status
- Media Storage: Use object storage (S3) for images/videos
- Push Notifications: Send to offline users via FCM/APNS
- End-to-End Encryption: Encrypt messages for privacy

Problem 3: Design a Video Streaming Service

Design YouTube/Netflix - upload, store, and stream videos.

Solution Approach:

- Upload Service: Handle large file uploads with resumable uploads
- Video Processing: Transcode to multiple resolutions/formats
- Content Delivery: Use CDN for video streaming
- Adaptive Bitrate: Adjust quality based on network conditions
- Metadata Storage: Store video info, comments, likes in database
- Recommendation Engine: Suggest videos using ML models

Problem 4: Design a Distributed Cache

Design a caching layer like Redis/Memcached.

Solution Approach:

- In-Memory Storage: Use hash map for O(1) access
- Eviction Policy: Implement LRU using doubly-linked list + hash map
- TTL Support: Expire keys after set time using lazy or active expiration
- Replication: Master-slave for read scaling and failover
- Sharding: Partition data across multiple nodes
- Persistence: Optional snapshot or append-only log

Quick Reference: System Design Checklist

Area	Questions to Ask
Scalability	How to handle 10x growth?
Availability	What happens if X fails?
Performance	What is the expected latency?
Security	How to authenticate/authorize?
Cost	What is the infrastructure cost?
Monitoring	How to detect and alert on issues?

System Design

Complete A to Z Guide

Created & Authored by

Mohammad Faiz

Copyright (c) 2026 Mohammad Faiz. All Rights Reserved.

No Permission to Use: No person or entity is permitted to use, copy, modify, merge, publish, distribute, sublicense, or sell any part of this content.

Personal Use Only: Only the original author Mohammad Faiz has the authority to use this content in any project or application.